**UNIT II CLASSES & OBJECTS**

Classes - Objects - Constructors - Overloading methods - Static and fixed methods - Inner Classes - String Class - Inheritance - Overriding methods - Using super- Abstract class.

## Classes and Objects in Java:

In Java, classes and objects are basic concepts of Object Oriented Programming (OOPs) that are used to represent real-world concepts and entities. The class represents a group of objects having similar properties and behavior. For example, the animal type **Dog** is a class while a particular dog named **Tommy** is an object of the **Dog** class.

### Java Classes

A class in Java is a set of objects which shares common characteristics/ behavior and common properties/ attributes.

### Properties of Java Classes

1. Class is not a real-world entity. It is just a template or blueprint or prototype from which objects are created.
2. Class does not occupy memory.
3. Class is a group of variables of different data types and a group of methods.
4. A Class in Java can contain:
   - Data member
   - Method
   - Constructor
   - Nested Class
   - Interface

### Class Declaration in Java

```
access_modifier class <class_name>
{
   data member;
   method;
   constructor;
   nested class;
   interface;
}
```

### Example of Java Class

```
class Student {
   // data member (also instance variable)
   int id;
   id=101
   // data member (also instance variable)
   String name;
  name='Java'
   public static void main(String args[])
   {
      // creating an object of
      // Student
      Student s1 = new Student();
      System.out.println(s1.id);
      System.out.println(s1.name);
   }
}
```

Output:
101
Java

**Components of Java Classes**

In general, class declarations can include these components, in order:

1. ***Modifiers****: A class can be public or has default access (Refer this for details).*
2. ***Class keyword:*** *class keyword is used to create a class.*
3. ***Class name:*** *The name should begin with an initial letter (capitalized by convention).*
4. ***Superclass(if any):*** *The name of the class's parent (superclass), if any, preceded by the keyword extends. A class can only extend (subclass) one parent.*
5. ***Interfaces(if any):*** *A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword implements. A class can implement more than one interface.*
6. ***Body:*** *The class body is surrounded by braces, { }.*

**Java Objects**

An object in Java is a basic unit of Object-Oriented Programming and represents real-life entities. Objects are the instances of a class that are created to use the attributes and methods of a class. A typical Java program creates many objects, which as you know, interact by invoking methods. An object consists of :

1. **State**: It is represented by attributes of an object. It also reflects the properties of an object.
2. **Behavior**: It is represented by the methods of an object. It also reflects the response of an object with other objects.
3. **Identity**: It gives a unique name to an object and enables one object to interact with other objects.

Example of an object: dog


**Create an Object**

create an object of clsobj, specify the class name, followed by the object name, and use the keyword new:

```
public class clsobj {
  int x = 5;

  public static void main(String[] args) {
    clsobj  myObj = new clsobj();
    System.out.println(myObj.x);
  }
}
```

Output:
5

**Access Methods With an Object**

Create a Car object named myCar. Call the fullThrottle() and speed() methods on the myCar object, and run the program:

```
// Create a Car class
public class Car{

    public void speed(int maxSpeed) {
    System.out.println("Max speed is: " + maxSpeed);
```

```
  }

  // Inside main, call the methods on the myCar object

  public static void main(String[] args) {
    Car myCar = new Car();   // Create a myCar object
    myCar.speed(200);          // Call the speed() method
  }
}
```

**Difference between Java Class and Objects**

| Class | Object |
|---|---|
| Class is the blueprint of an object. It is used to create objects. | An object is an instance of the class. |
| No memory is allocated when a class is declared. | Memory is allocated as soon as an object is created. |
| A class is a group of similar objects. | An object is a real-world entity such as a book, car, etc. |
| Class is a logical entity. | An object is a physical entity. |
| A class can only be declared once. | Objects can be created many times as per requirement. |
| An example of class can be a car. | Objects of the class car can be BMW, Mercedes, Ferrari, etc. |

**Remember that..**
The dot (.) is used to access the object's attributes and methods.
To call a method in Java, write the method name followed by a set of parentheses **()**, followed by a semicolon (;).
A class must have a matching filename (Car and **Car.java**).

## Constructors in Java

- In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.
- It is a special type of method which is used to initialize the object.
- Every time an object is created using the new() keyword, at least one constructor is called.
- It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.
- There are two types of constructors in Java: **no-arg constructor, and parameterized** constructor.

### Rules for creating Java constructor

- Constructor name must be the same as its class name
- A Constructor must have no explicit return type
- A Java constructor cannot be abstract, static, final, and synchronized

### Example of default constructor

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked at the time of object creation.

```
class Bike1{
//creating a default constructor
Bike1(){System.out.println("Bike is created");}
//main method
public static void main(String args[]){
//calling a default constructor
Bike1 b=new Bike1();
}
}
```

**Compile by: javac Bike1.java**
**Run by: java Bike1**
Bike is created

**Example of parameterized constructor**
In this example, we have created the constructor of Student class that have two parameters. We can have any number of parameters in the constructor

```
class Student{
   int id;
   String name;
   //creating a parameterized constructor
   Student(int i,String n){
   id = i;
   name = n;
   }
   //method to display the values
   void display(){System.out.println(id+" "+name);}

   public static void main(String args[]){
  //creating objects and passing values
   Student  s1 = new Student(111,"Karan");
   Student  s2 = new Student(222,"Aryan");
   //calling method to display the values of object
   s1.display();
   s2.display();
  }
}
```

**Constructor Overloading in Java**
In Java, a constructor is just like a method but without return type. It can also be overloaded like Java methods.
Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

**Example of Constructor Overloading**

```
class Student{
   int id;
   String name;
   int age;
   //creating two arg constructor
   Student(int i,String n){
   id = i;
   name = n;
   }
   //creating three arg constructor
   Student(int i,String n,int a){
```

```
    id = i;
    name = n;
    age=a;
    }
    void display(){
System.out.println(id+" "+name+" "+age);
}

    public static void main(String args[]){
    Student s1 = new Student(111,"Karan");
    Student s2 = new Student(222,"Aryan",25);
    s1.display();
    s2.display();
   }
}
```

**Compile by: javac Student.java**

**Run by: java Student**

```
111 Karan 0
222 Aryan 25
```

**Difference between constructor and method in Java**

| Java Constructor | Java Method |
|---|---|
| A constructor is used to initialize the state of an object. | A method is used to expose the behavior of an object. |
| A constructor must not have a return type. | A method must have a return type. |
| The constructor is invoked implicitly. | The method is invoked explicitly. |
| The Java compiler provides a default constructor if you don't have any constructor in a class. | The method is not provided by the compiler in any case. |
| The constructor name must be same as the class name. | The method name may or may not be same as the class name. |

**Java Copy Constructor**

There is no copy constructor in Java. However, we can copy the values from one object to another like copy constructor in C++.

There are many ways to copy the values of one object into another in Java. They are:

- By constructor
- By assigning the values of one object into another

In this example, we are going to copy the values of one object into another using Java constructor.

```
//Java program to initialize the values from one object to another object.
class Student6{
   int id;
   String name;
```

```
   //constructor to initialize integer and string
   Student6(int i,String n){
   id = i;
   name = n;
   }
   //constructor to initialize another object
   Student6(Student6 s){
   id = s.id;
   name =s.name;
   }
   void display(){
System.out.println(id+" "+name);
}

   public static void main(String args[]){
   Student6 s1 = new Student6(111,"Karan");
   Student6 s2 = new Student6(s1);
   s1.display();
   s2.display();
  }
}
```

Output:
111 Karan
111 Karan

**Copying values without constructor**

We can copy the values of one object into another by assigning the objects values to another object. In this case, there is no need to create the constructor.

```
class Student7{
int id;
String name;
Student7(int i,String n){
id = i;
name = n;
}
Student7(){}
void display(){
System.out.println(id+" "+name);
}

public static void main(String args[]){
Student7 s1 = new Student7(111,"Karan");
Student7 s2 = new Student7();
```

```
s2.id=s1.id;
s2.name=s1.name;
s1.display();
s2.display();
}
}
```

Output:

```
111 Karan
111 Karan
```

---

## Java static keyword

The **static keyword** in Java is used for memory management mainly. We can apply static keyword with variables, methods, blocks and nested classes. The static keyword belongs to the class than an instance of the class.

### 1) Java static variable

If you declare any variable as static, it is known as a static variable.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- The static variable gets memory only once in the class area at the time of class loading.

### Advantages of static variable

It makes your program **memory efficient** (i.e., it saves memory).

```
1. class Student{
2.     int rollno;
3.     String name;
4.     String college="XXX";
5. }
```

Suppose there are 500 students in my college, now all instance data members will get memory each time when the object is created. All students have its unique rollno and name, so instance data member is good in such case. Here, "college" refers to the common property of all objects. If we make it static, this field will get the memory only once.

*Java static property is shared to all objects.*

### Example of static variable

```
//Java Program to demonstrate the use of static variable
class Student{
  int rollno;//instance variable
  String name;
  static String college ="XXX";//static variable
```

```
  //constructor
  Student(int r, String n){
  rollno = r;
  name = n;
  }
  //method to display the values
  void display (){System.out.println(rollno+" "+name+" "+college);}
}
//Test class to show the values of objects
public class TestStaticVariable1{
 public static void main(String args[]){
 Student s1 = new Student(111,"Karan");
 Student s2 = new Student(222,"Aryan");
 //we can change the college of all objects by the single line of code
 //Student.college="XXX";
 s1.display();
 s2.display();
 }
}
```

Output:
111 Karan XXX
222 Aryan XXX

**Static Method in Java With Examples**

The static keyword is used to construct methods that will exist regardless of whether or not any instances of the class are generated. Any method that uses the static keyword is referred to as a static method.

**Features of static method:**

- A static method in Java is a method that is part of a class rather than an instance of that class.
- Every instance of a class has access to the method.
- Static methods have access to class variables (static variables) without using the class's object (instance).
- Only static data may be accessed by a static method. It is unable to access data that is not static (instance variables).
- In both static and non-static methods, static methods can be accessed directly.

**Syntax to declare the static method:**

```
Access_modifier static void methodName()
{
    // Method body.
}
```

The name of the class can be used to invoke or access static methods.

**Syntax to call a static method:**

```
className.methodName();
```

***Example 1: The static method does not have access to the instance variable***

The JVM runs the static method first, followed by the creation of class instances. Because no objects are accessible when the static method is used. A static method does not have access to instance variables. As a result, a static method can't access a class's instance variable.

```
import java.io.*;

public class staex {
   // static variable
   static int a = 40;

   // instance variable
   int b = 50;

   void simpleDisplay()
   {
      System.out.println(a);
      System.out.println(b);
   }

   // Declaration of a static method.
   static void staticDisplay()
   {
     System.out.println(a);
   }

   // main method
   public static void main(String[] args)
   {
      staex obj = new staex();
      obj.simpleDisplay();

      // Calling static method.
      staticDisplay();
   }
}
```

**Output**

```
40
50
40
```

***Example 2: In both static and non-static methods, static methods are directly accessed.***

```
import java.io.*;

public class StaticExample {

   static int num = 100;
   static String str = "Welcome";

   // This is Static method
   static void display()
   {
      System.out.println("static number is " + num);
      System.out.println("static string is " + str);
   }
```

```
    // non-static method
    void nonstatic()
    {
        // our static method can accessed
        // in non static method
        display();
    }

    // main method
    public static void main(String args[])
    {
        StaticExample obj = new StaticExample();

        // This is object to call non static function
        obj.nonstatic();

        // static method can called
        // directly without an object
        display();
    }
}
```

**Output**

```
static number is 100
static string is Welcome
static number is 100
static string is Welcome
```

***Why use Static Methods?***

1. To access and change static variables and other non-object-based static methods.
2. Utility and assist classes frequently employ static methods.

***Restrictions in Static Methods:***

1. Non-static data members or non-static methods cannot be used by static methods, and static methods cannot call non-static methods directly.
2. In a static environment, this and super aren't allowed to be used.

## Java Inner Classes

In Java, it is also possible to nest classes (a class within a class). The purpose of nested classes is to group classes that belong together, which makes your code more readable and maintainable.

To access the inner class, create an object of the outer class, and then create an object of the inner class:

```
class OuterClass {
  int x = 10;

  class InnerClass {
    int y = 5;
  }
}
public class Main {
  public static void main(String[] args) {
    OuterClass myOuter = new OuterClass();
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
    System.out.println(myInner.y + myOuter.x);
  }
}
```

**Output**
**15**

**Private Inner Class:**

Unlike a "regular" class, an inner class can be private or protected. If you don't want outside objects to access the inner class, declare the class as private:

```
class OuterClass {
int x = 10;
private class InnerClass {
int y = 5;       }}
public class Main {
public static void main(String[] args) {
OuterClass myOuter = new OuterClass();
OuterClass.InnerClass myInner = myOuter.new InnerClass();
System.out.println(myInner.y + myOuter.x);
}
}
```

If you try to access a private inner class from an outside class, an error occurs:

**Main.java:13: error: OuterClass.InnerClass has private access in OuterClass**
**OuterClass.InnerClass myInner = myOuter.new InnerClass();**

**Static Inner Class:-**An inner class can also be static, which means that you can access it without creating an object of the outer class:

**Example**

```
class OuterClass {
int x = 10;
static class InnerClass {
int y = 5;
}public class Main {
public static void main(String[] args) {
OuterClass.InnerClass myInner = new OuterClass.InnerClass();
System.out.println(myInner.y);
}
}

// Outputs 5
```

**Note:** just like static attributes and methods, a static inner class does not have access to members of the outer class.

**Access Outer Class From Inner Class**

One advantage of inner classes, is that they can access attributes and methods of the outer class:

**Example**

```
class OuterClass {
  int x = 10;
  class InnerClass {
    public int myInnerMethod() {
      return x;
    }}}
```

```
public class Main {
  public static void main(String[] args) {
    OuterClass myOuter = new OuterClass();
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
    System.out.println(myInner.myInnerMethod());
  }
}

// Outputs 10
```

**Java String**

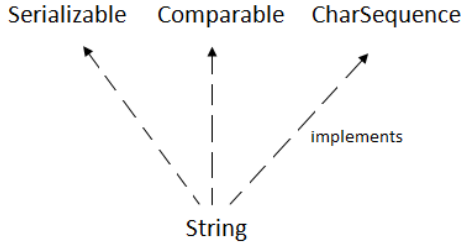
In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java string.

**For example:**

**char**[] ch={'j','a','v','a','t', **Java String** class provides a lot of methods to perform operations on strings such as compare(), concat(), equals(), split(), length(), replace(), compareTo(), intern(), substring() etc.

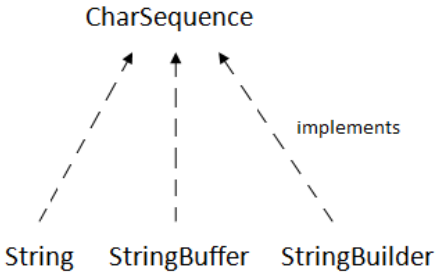
The java.lang.String class implements *Serializable*, *Comparable* and *CharSequence* interfaces.
'p','o','i','n','t'};

String s=**new** String(ch);
is same as:
String s="javatpoint";




## CharSequence Interface

The CharSequence interface is used to represent the sequence of characters.
String, StringBuffer and StringBuilder classes implement it. It means, we can create strings in Java by using these three classes.




The Java String is immutable which means it cannot be changed. Whenever we change any string, a new instance is created. For mutable strings, you can use StringBuffer and StringBuilder classes.
We will discuss immutable string later. Let's first understand what String in Java is and how to create the String object.

## What is String in Java?

Generally, String is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The java.lang.String class is used to create a string object.

## How to create a string object?

There are two ways to create String object:
By string literal
By new keyword

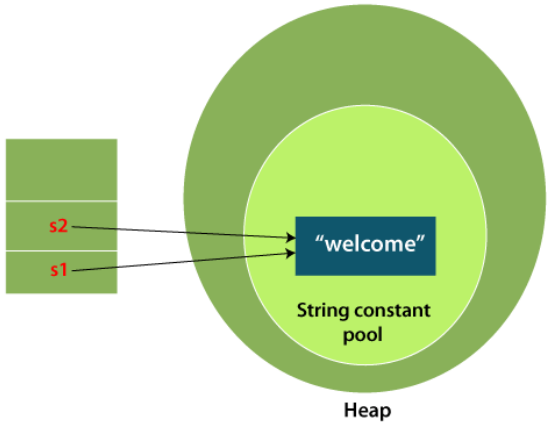
### 1) String Literal

Java String literal is created by using double quotes.

For Example:
String s="welcome";

Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool.

For example:
String s1="Welcome";
String s2="Welcome";//It doesn't create a new instance




In the above example, only one object will be created. Firstly, JVM will not find any string object with the value "Welcome" in string constant pool that is why it will create a new object. After that it will find the string with the value "Welcome" in the pool, it will not create a new object but will return the reference to the same instance.
Note: String objects are stored in a special memory area known as the "string constant pool".

## Why Java uses the concept of String literal?

To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

## 2) By new keyword

String s=**new** String("Welcome");//creates two objects and one reference variable
In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in a heap (non-pool).

## Java String Example

**StringExample.java**

```
public class StringExample{
public static void main(String args[]){
String s1="java";//creating string by Java string literal
char ch[]={'s','t','r','i','n','g','s'};
String s2=new String(ch);//converting char array to string
String s3=new String("example");//creating Java string by new keyword
System.out.println(s1);
System.out.println(s2);
System.out.println(s3);
}}
```

**Output:**

```
java
strings
example
```

The above code, converts a ***char*** array into a **String** object. And displays the String objects ***s1, s2***, and ***s3*** on console using ***println()*** method.

## Java String class methods

The java.lang.String class provides many useful methods to perform operations on sequence of char values.

| No. | Method | Description |
|---|---|---|
| 1 | char charAt(int index) | It returns char value for the particular index |
| 2 | int length() | It returns string length |
| 3 | static String format(String format, Object... args) | It returns a formatted string. |
| 4 | static String format(Locale l, String format, Object... args) | It returns formatted string with given locale. |
| 5 | String substring(int beginIndex) | It returns substring for given begin index. |
| 6 | String substring(int beginIndex, int endIndex) | It returns substring for given begin index and end index. |
| 7 | boolean contains(CharSequence s) | It returns true or false after matching the sequence of char value. |
| 8 | static String join(CharSequence delimiter, CharSequence... elements) | It returns a joined string. |
| 9 | static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements) | It returns a joined string. |
| 10 | boolean equals(Object another) | It checks the equality of string with the given object. |
| 11 | boolean isEmpty() | It checks if string is empty. |
| 12 | String concat(String str) | It concatenates the specified string. |
| 13 | String replace(char old, char new) | It replaces all occurrences of the specified char value. |
| 14 | String replace(CharSequence old, CharSequence new) | It replaces all occurrences of the specified CharSequence. |
| 15 | static String equalsIgnoreCase(String another) | It compares another string. It doesn't check case. |
| 16 | String[] split(String regex) | It returns a split string matching regex. |
| 17 | String[] split(String regex, int limit) | It returns a split string matching regex and limit. |
| 18 | String intern() | It returns an interned string. |

| | | |
|---|---|---|
| 19 | int indexOf(int ch) | It returns the specified char value index. |
| 20 | int indexOf(int ch, int fromIndex) | It returns the specified char value index starting with given index. |
| 21 | int indexOf(String substring) | It returns the specified substring index. |
| 22 | int indexOf(String substring, int fromIndex) | It returns the specified substring index starting with given index. |
| 23 | String toLowerCase() | It returns a string in lowercase. |
| 24 | String toLowerCase(Locale l) | It returns a string in lowercase using specified locale. |
| 25 | String toUpperCase() | It returns a string in uppercase. |
| 26 | String toUpperCase(Locale l) | It returns a string in uppercase using specified locale. |
| 27 | String trim() | It removes beginning and ending spaces of this string. |

**Java String Class Methods**

The **java.lang.String** class provides a lot of built-in methods that are used to manipulate **string in Java**. By the help of these methods, we can perform operations on String objects such as trimming, concatenating, converting, comparing, replacing strings etc.

Java String is a powerful concept because everything is treated as a String if you submit any form in window based, web based or mobile application.

**Java String toUpperCase() and toLowerCase() method**

The Java String toUpperCase() method converts this String into uppercase letter and String toLowerCase() method into lowercase letter.

```
1. public class Stringoperation1
2. {
3. public static void main(String ar[])
4. {
5. String s="Sachin";
6. System.out.println(s.toUpperCase());//SACHIN
7. System.out.println(s.toLowerCase());//sachin
8. System.out.println(s);//Sachin(no change in original)
9. }
10. }
```

Output:
SACHIN
sachin
Sachin

**Java String trim() method**

The String class trim() method eliminates white spaces before and after the String.

```
1. public class Stringoperation2
2. {
3. public static void main(String ar[])
4. {
5. String s="  Sachin  ";
6. System.out.println(s);//  Sachin
7. System.out.println(s.trim());//Sachin
8. }
9. }
```

Output:
Sachin
Sachin

**Java String startsWith() and endsWith() method**

The method startsWith() checks whether the String starts with the letters passed as arguments and endsWith() method checks whether the String ends with the letters passed as arguments.

```
1. public class Stringoperation3
2. {
3. public static void main(String ar[])
4. {
5. String s="Sachin";
6.  System.out.println(s.startsWith("Sa"));//true
7.  System.out.println(s.endsWith("n"));//true
8. }
9. }
```

Output:
true
true

**Java String charAt() Method**

The String class charAt() method returns a character at specified index.

```
1. public class Stringoperation4
2. {
3. public static void main(String ar[])
4. {
```

```
5. String s="Sachin";
6. System.out.println(s.charAt(0));//S
7. System.out.println(s.charAt(3));//h
8. }
9. }
```

Output:

S
h

**Java String length() Method**

The String class length() method returns length of the specified String.

```
1. public class Stringoperation5
2. {
3. public static void main(String ar[])
4. {
5. String s="Sachin";
6. System.out.println(s.length());//6
7. }
8. }
```

Output: 6

**Java String intern() Method**

A pool of strings, initially empty, is maintained privately by the class String.

When the intern method is invoked, if the pool already contains a String equal to this String object as determined by the equals(Object) method, then the String from the pool is returned. Otherwise, this String object is added to the pool and a reference to this String object is returned.

```
1. public class Stringoperation6
2. {
3. public static void main(String ar[])
4. {
5. String s=new String("Sachin");
6. String s2=s.intern();
7. System.out.println(s2);//Sachin
8. }
9. }
```

Output:

Sachin

**Java String valueOf() Method**

The String class valueOf() method coverts given type such as int, long, float, double, boolean, char and char array into String.

```
1. public class Stringoperation7
2. {
3. public static void main(String ar[])
4. {
5. int a=10;
6. String s=String.valueOf(a);
7. System.out.println(s+10);
8. }
9. }
```

Output:

1010

**Java String replace() Method**

The String class replace() method replaces all occurrence of first sequence of character with second sequence of character.

```
1. public class Stringoperation8
2. {
3. public static void main(String ar[])
4. {
5. String s1="Java is a programming language. Java is a platform. Java is an Island.";
6. String replaceString=s1.replace("Java","Kava");//replaces all occurrences of "Java" to "Kava"
7. System.out.println(replaceString);
8. }
9. }
```

Output:

Kava is a programming language. Kava is a platform. Kava is an Island.

## Java String compareTo()

The **Java String class compareTo()** method compares the given string with the current string lexicographically. It returns a positive number, negative number, or 0.

It compares strings on the basis of the Unicode value of each character in the strings.

If the first string is lexicographically greater than the second string, it returns a positive number (difference of character value). If the first string is less than the second string lexicographically, it returns a negative number, and if the first string is lexicographically equal to the second string, it returns 0.

1. **if** s1 > s2, it returns positive number
2. **if** s1 < s2, it returns negative number
3. **if** s1 == s2, it returns 0

```
1. public class CompareToExample{
```

```
2. public static void main(String args[]){
3. String s1="hello";
4. String s2="hello";
5. String s3="meklo";
6. String s4="hemlo";
7. String s5="flag";
8. System.out.println(s1.compareTo(s2));//0 because both are equal
9. System.out.println(s1.compareTo(s3));//-5 because "h" is 5 times lower than "m"
10. System.out.println(s1.compareTo(s4));//-1 because "l" is 1 times lower than "m"
11. System.out.println(s1.compareTo(s5));//2 because "h" is 2 times greater than "f"
12. }}
```

Output:

```
0
-5
-1
2
```

## Java String concat

The **Java String class concat()** method *combines specified string at the end of this string*. It returns a combined string. It is like appending another string.

```
1. public class ConcatExample{
2. public static void main(String args[]){
3. String s1="java string";
4. // The string s1 does not get changed, even though it is invoking the method
5. // concat(), as it is immutable. Therefore, the explicit assignment is required here.
6. s1.concat("is immutable");
7. System.out.println(s1);
8. s1=s1.concat(" is immutable so assign it explicitly");
9. System.out.println(s1);
10. }}
```

Output:

```
java string
java string is immutable so assign it explicitly
```

## Java String contains()

The **Java String class contains()** method searches the sequence of characters in this string. It returns *true* if the sequence of char values is found in this string otherwise returns *false*.

```
1. class ContainsExample{
2. public static void main(String args[]){
3. String name="what do you know about me";
```

```
4. System.out.println(name.contains("do you know"));
5. System.out.println(name.contains("about"));
6. System.out.println(name.contains("hello"));
7. }}
```

Output:

```
true
true
false
```

## Java String equals()

The **Java String class equals()** method compares the two given strings based on the content of the string. If any character is not matched, it returns false. If all characters are matched, it returns true.

The String equals() method overrides the equals() method of the Object class.

```
1. public class EqualsExample{
2. public static void main(String args[]){
3. String s1="javatpoint";
4. String s2="javatpoint";
5. String s3="JAVATPOINT";
6. String s4="python";
7. System.out.println(s1.equals(s2));//true because content and case is same
8. System.out.println(s1.equals(s3));//false because case is not same
9. System.out.println(s1.equals(s4));//false because content is not same
10. }}
```

Output:

```
true
false
false
```

## Java String equalsIgnoreCase()

The Java **String class equalsIgnoreCase()** method compares the two given strings on the basis of the content of the string irrespective of the case (lower and upper) of the string. It is just like the equals() method but doesn't check the case sensitivity. If any character is not matched, it returns false, else returns true.

```
1. public class EqualsIgnoreCaseExample{
2. public static void main(String args[]){
3. String s1="javatpoint";
4. String s2="javatpoint";
5. String s3="JAVATPOINT";
6. String s4="python";
7. System.out.println(s1.equalsIgnoreCase(s2));//true because content and case both are same
8. System.out.println(s1.equalsIgnoreCase(s3));//true because case is ignored
9. System.out.println(s1.equalsIgnoreCase(s4));//false because content is not same
```

```
10. }}
```

Output:

true
true
false

## Java String join()

The **Java String class join()** method returns a string joined with a given delimiter. In the String join() method, the delimiter is copied for each element.

```
1. public class StringJoinExample{
2. public static void main(String args[]){
3. String joinString1=String.join("-","welcome","to","javatpoint");
4. System.out.println(joinString1);
5. }}
```

Output:

welcome-to-javatpoint

## Java String split()

```
public class SplitExample{
public static void main(String args[]){
String s1="java string split method by javatpoint";
String[] words=s1.split("s");//splits the string based on string
//using java foreach loop to print elements of string array
for(String w:words){
System.out.println(w);
}
}}
```

Output:

```
java
tring
plit method by javatpoint
```

## Substring in Java

A part of String is called **substring**. In other words, substring is a subset of another String. Java String class provides the built-in substring() method that extract a substring from the given string by using the index values passed as an argument. In case of substring() method startIndex is inclusive and endIndex is exclusive.

Suppose the string is "**computer**", then the substring will be com, compu, ter, etc.

You can get substring from the given String object by one of the two methods:

1. **Public String substring(int startIndex):**
   This method returns new String object containing the substring of the given string from specified startIndex (inclusive). The method throws an IndexOutOfBoundException when the startIndex is larger than the length of String or less than zero.

2. **public String substring(int startIndex, int endIndex):** This method returns new String object containing the substring of the given string from specified startIndex to endIndex. The method throws an IndexOutOfBoundException when the startIndex is less than zero or startIndex is greater than endIndex or endIndex is greater than length of String.

```
1. public class TestSubstring{
2.  public static void main(String args[]){
3.  String s="SachinTendulkar";
4.  System.out.println("Original String: " + s);
5.  System.out.println("Substring starting from index 6: " +s.substring(6));//Tendulkar
6.  System.out.println("Substring starting from index 0 to 6: "+s.substring(0,6)); //Sachin
7.  }
8. }
```

Output:

```
Original String: SachinTendulkar
Substring starting from index 6: Tendulkar
Substring starting from index 0 to 6: Sachin
```

**StringBuffer Class**

StringBuffer is a peer class of String that provides much of the functionality of strings. The string represents fixed-length, immutable character sequences while StringBuffer represents growable and writable character sequences. StringBuffer may have characters and substrings inserted in the middle or appended to the end. It will automatically grow to make room for such additions and often has more characters preallocated than are actually needed, to allow room for growth. In order to create a string buffer, an object needs to be created, (i.e.), if we wish to create a new string buffer with name str, then:

```
StringBuffer str = new StringBuffer();
```

**Here are some important features and methods of the StringBuffer class:**

- StringBuffer objects are mutable, meaning that you can change the contents of the buffer without creating a new object.
- The initial capacity of a StringBuffer can be specified when it is created, or it can be set later with the ensureCapacity() method.
- The append() method is used to add characters, strings, or other objects to the end of the buffer.
- The insert() method is used to insert characters, strings, or other objects at a specified position in the buffer.
- The delete() method is used to remove characters from the buffer.
- The reverse() method is used to reverse the order of the characters in the buffer.

**1) StringBuffer Class append() Method**

The append() method concatenates the given argument with this String.

```
1. class StringBufferExample{
2. public static void main(String args[]){
3. StringBuffer sb=new StringBuffer("Hello ");
```

```
4. sb.append("Java");//now original string is changed
5. System.out.println(sb);//prints Hello Java
6. } }
```

Output:

**Hello World**

**2) StringBuffer insert() Method**

The insert() method inserts the given String with this string at the given position.

```
1. class StringBufferExample2{
2. public static void main(String args[]){
3. StringBuffer sb=new StringBuffer("Hello ");
4. sb.insert(1,"Java");//now original string is changed
5. System.out.println(sb);//prints HJavaello
6. }
7. }
```

Output:
HJavaello

**3) StringBuffer replace() Method**

The replace() method replaces the given String from the specified beginIndex and endIndex.

```
1. class StringBufferExample3{
2. public static void main(String args[]){
3. StringBuffer sb=new StringBuffer("Hello");
4. sb.replace(1,3,"Java");
5. System.out.println(sb);//prints HJavalo
6. }
7. }
```

Output:
HJavalo

**4) StringBuffer delete() Method**

The delete() method of the StringBuffer class deletes the String from the specified beginIndex to endIndex.

```
1. class StringBufferExample4{
2. public static void main(String args[]){
3. StringBuffer sb=new StringBuffer("Hello");
4. sb.delete(1,3);
5. System.out.println(sb);//prints Hlo
```

```
6. }
7. }
```

Output:

Hlo

**5) StringBuffer reverse() Method**

The reverse() method of the StringBuilder class reverses the current String.

```
1. class StringBufferExample5{
2. public static void main(String args[]){
3. StringBuffer sb=new StringBuffer("Hello");
4. sb.reverse();
5. System.out.println(sb);//prints olleH
6. }
7. }
```

Output:

olleH

**StringBuffer capacity() Method**

The capacity() method of the StringBuffer class returns the current capacity of the buffer. The default capacity of the buffer is 16. If the number of character increases from its current capacity, it increases the capacity by (oldcapacity*2)+2. For example if your current capacity is 16, it will be (16*2)+2=34.

```
1. class StringBufferExample6{
2. public static void main(String args[]){
3. StringBuffer sb=new StringBuffer();
4. System.out.println(sb.capacity());//default 16
5. sb.append("Hello");
6. System.out.println(sb.capacity());//now 16
7. sb.append("java is my favourite language");
8. System.out.println(sb.capacity());//now (16*2)+2=34 i.e (oldcapacity*2)+2
9. }
10. }
```

Output:

16
16
34

**There are several advantages of using StringBuffer over regular String objects in Java:**

- **Mutable:** StringBuffer objects are mutable, which means that you can modify the contents of the object after it has been created. In contrast, String objects are immutable, which means that you cannot change the contents of a String once it has been created.
- **Efficient**: Because StringBuffer objects are mutable, they are more efficient than creating new String objects each time you need to modify a string. This is especially true if you need to modify

a string multiple times, as each modification to a String object creates a new object and discards the old one.

**Java StringBuilder Examples**

Similar to StringBuffer, the StringBuilder in Java represents a mutable sequence of characters. Since the String Class in Java creates an immutable sequence of characters, the StringBuilder class provides an alternative to String Class, as it creates a mutable sequence of characters. The function of StringBuilder is very much similar to the StringBuffer class, as both of them provide an alternative to String Class by making a mutable sequence of characters. Similar to StringBuffer, in order to create a new string with the name str, we need to create an object of StringBuilder, (i.e.):

```
StringBuilder str = new StringBuilder();
```

Let's see the examples of different methods of StringBuilder class.

**1) StringBuilder append() method**

The StringBuilder append() method concatenates the given argument with this String.

```
1. class StringBuilderExample{
2. public static void main(String args[]){
3. StringBuilder sb=new StringBuilder("Hello ");
4. sb.append("Java");//now original string is changed
5. System.out.println(sb);//prints Hello Java
6. }
7. }
```

**Output:**

```
Hello Java
```

**2) StringBuilder insert() method**

The StringBuilder insert() method inserts the given string with this string at the given position.

```
1. class StringBuilderExample2{
2. public static void main(String args[]){
3. StringBuilder sb=new StringBuilder("Hello ");
4. sb.insert(1,"Java");//now original string is changed
5. System.out.println(sb);//prints HJavaello
6. }
7. }
```

**Output:**

```
HJavaello
```

**3) StringBuilder replace() method**

The StringBuilder replace() method replaces the given string from the specified beginIndex and endIndex.

```
1. class StringBuilderExample3{
2. public static void main(String args[]){
3. StringBuilder sb=new StringBuilder("Hello");
4. sb.replace(1,3,"Java");
5. System.out.println(sb);//prints HJavalo
6. }
7. }
```

**Output:**

```
HJavalo
```

**4) StringBuilder delete() method**

The delete() method of StringBuilder class deletes the string from the specified beginIndex to endIndex.

```
1. class StringBuilderExample4{
2. public static void main(String args[]){
3. StringBuilder sb=new StringBuilder("Hello");
4. sb.delete(1,3);
5. System.out.println(sb);//prints Hlo
6. }
7. }
```

**Output:**

```
Hlo
```

**5) StringBuilder reverse() method**

The reverse() method of StringBuilder class reverses the current string.

```
1. class StringBuilderExample5{
2. public static void main(String args[]){
3. StringBuilder sb=new StringBuilder("Hello");
4. sb.reverse();
5. System.out.println(sb);//prints olleH
6. }
7. }
```

Output:
olleH

**6) StringBuilder capacity() method**

The capacity() method of StringBuilder class returns the current capacity of the Builder. The default capacity of the Builder is 16. If the number of character increases from its current capacity, it increases the capacity by (oldcapacity*2)+2. For example if your current capacity is 16, it will be (16*2)+2=34.

```
1. class StringBuilderExample6{
2. public static void main(String args[]){
3. StringBuilder sb=new StringBuilder();
4. System.out.println(sb.capacity());//default 16
5. sb.append("Hello");
6. System.out.println(sb.capacity());//now 16
7. sb.append("Java is my favourite language");
8. System.out.println(sb.capacity());//now (16*2)+2=34 i.e (oldcapacity*2)+2
9. }
10. }
```

Output:
16
16
34

| Sr. No. | Key | String Buffer | String Builder |
|---|---|---|---|
| 1 | Basic | StringBuffer was introduced with the initial release of Java | It was introduced in Java 5 |
| 2 | Synchronized | It is synchronized | It is not synchronized |
| 3 | Performance | It is thread-safe. So, multiple threads can't access at the same time, therefore, it is slow. | It is not thread-safe hence faster than String Buffer. |
| 4 | Mutable | It is mutable. We can modify string without creating an object. | It is also mutable |
| 5 | Storage | Heap | Heap |

**Inheritance in Java** is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.

Inheritance represents the **IS-A relationship** which is also known as a *parent-child* relationship.

**Why use inheritance in java**

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

**Terms used in Inheritance**

**Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.

**Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.

**Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.

**Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

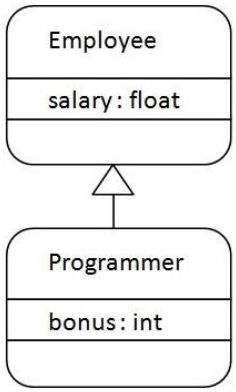
**The syntax of Java Inheritance**

```
class Subclass-name extends Superclass-name
{
  //methods and fields
}
```

The **extends keyword** indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

In the terminology of Java, a class which is inherited is called a parent or superclass, and the new class is called child or subclass

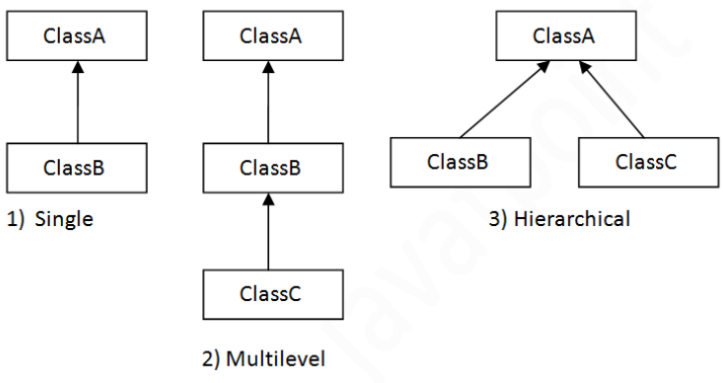
Java Inheritance Example




As displayed in the above figure, Programmer is the subclass and Employee is the superclass. The relationship between the two classes is **Programmer IS-A Employee**. It means that Programmer is a type of Employee.

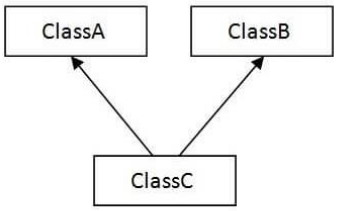
**Types of inheritance in java**

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical. In java programming, multiple and hybrid inheritance is supported through interface only. We will learn about interfaces later.

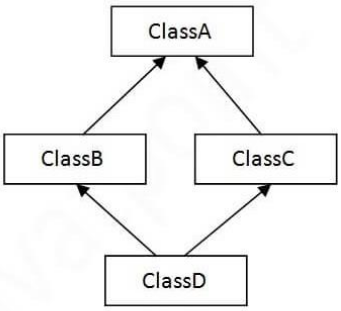


*Note: Multiple inheritance is not supported in Java through class.*

When one class inherits multiple classes, it is known as multiple inheritance. For Example:




4) Multiple




5) Hybrid

## Single Inheritance Example

When a class inherits another class, it is known as a *single inheritance*. In the example given below, Child class inherits the parent class, so there is the single inheritance.

```
class parent{
void sing(){System.out.println("singing...");}
}
class child extends parent{
void dance(){System.out.println("dancing...");}
}
class TestInheritance{
public static void main(String args[]){
child c=new child();
c.dance();
c.sing();
}
}
```

**Output:**

dancing…
singing…

## Multilevel Inheritance Example

- When there is a chain of inheritance, it is known as *multilevel inheritance*. As you can see in the example given below, grandchild class inherits the child class which again inherits the parent class, so there is a multilevel inheritance.

```
class parent{
void sing(){System.out.println("singing...");}
}
class child extends parent{
void dance(){System.out.println("dancing...");}
}
class gradchild extends child{
void art(){System.out.println("Drawing...");}
}
class TestInheritance2{
public static void main(String args[]){
grandchild d=new grandchild();
d.art();
d.dance();
```

```
d.sing();
}}
```
Output:
Drawing...
dancing...
singing...

**Hierarchical Inheritance Example**

When two or more classes inherits a single class, it is known as *hierarchical inheritance*. In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

```
1. class Animal{
2. void eat(){System.out.println("eating...");}
3. }
4. class Dog extends Animal{
5. void bark(){System.out.println("barking...");}
6. }
7. class Cat extends Animal{
8. void meow(){System.out.println("meowing...");}
9. }
10. class TestInheritance3{
11. public static void main(String args[]){
12. Cat c=new Cat();
13. c.meow();
14. c.eat();
15. //c.bark();//C.T.Error
16. }}
```
Output:
meowing...
eating...

**Why multiple inheritance is not supported in java?**

To reduce the complexity and simplify the language, multiple inheritance is not supported in java.

Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.

Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

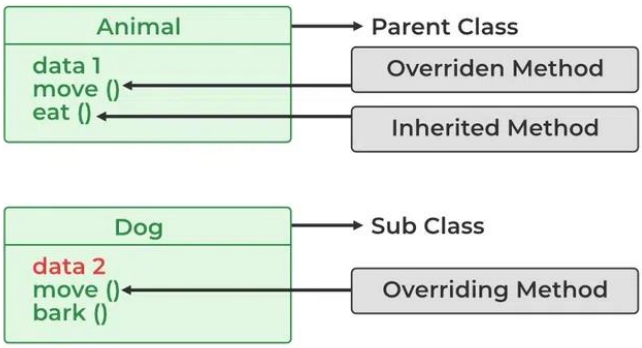
```
1. class A{
2. void msg(){System.out.println("Hello");}  }
3. class B{
4. void msg(){System.out.println("Welcome");}
5. }
6. class C extends A,B{//suppose if it were
7.
```

```
8.  public static void main(String args[]){
9.    C obj=new C();
10.   obj.msg();//Now which msg() method would be invoked?
11. } }
```

Compile Time Error

**Overiding in Java:**

In Java, Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a subclass has the same name, the same parameters or signature, and the same return type(or sub-type) as a method in its super-class, then the method in the subclass is said to *override* the method in the super-class.




Method overriding is one of the ways by which Java achieves Run Time Polymorphism. The version of a method that is executed will be determined by the object that is used to invoke it. If an object of a parent class is used to invoke the method, then the version in the parent class will be executed, but if an object of the subclass is used to invoke the method, then the version in the child class will be executed. In other words, *it is the type of the object being referred to* (not the type of the reference variable) that determines which version of an overridden method will be executed

**Example of Method Overriding in Java**

Below is the implementation of the Java Method Overriding:

```
// Base Class
class Parent {
   void show() { System.out.println("Parent's show()"); }
}
// Inherited class
class Child extends Parent {
   // This method overrides show() of Parent Override
   void show()
   {
      System.out.println("Child's show()");
   }
}
// Driver class
class Main {
   public static void main(String[] args)
   {
      // If a Parent type reference refers
      // to a Parent object, then Parent's
```

```
        // show is called
        Parent obj1 = new Parent();
        obj1.show();

        // If a Parent type reference refers
        // to a Child object Child's show()
        // is called. This is called RUN TIME
        // POLYMORPHISM.
        Parent obj2 = new Child();
        obj2.show();
    }
}
```

Output
Parent's show()
Child's show()

**1. Overriding and Access Modifiers**

The access modifier for an overriding method can allow more, but not less, access than the overridden method. For example, a protected instance method in the superclass can be made public, but not private, in the subclass. Doing so will generate a compile-time error.

```
class Parent {
    // private methods are not overridden
    private void m1()
    {
        System.out.println("From parent m1()");
    }

    protected void m2()
    {
        System.out.println("From parent m2()");
    }
}
class Child extends Parent {
    // new m1() method
    // unique to Child class
    private void m1()
    {
        System.out.println("From child m1()");
    }
     // overriding method
    // with more accessibility
    @Override public void m2()
    {
        System.out.println("From child m2()");
    }
}
 // Driver class
class Main {
    public static void main(String[] args)
    {
        Parent obj1 = new Parent();
        obj1.m2();
        Parent obj2 = new Child();
        obj2.m2();
    }
```

```
}
```
Output
From parent m2()
From child m2()

**2. Final methods can not be overridden**

If we don't want a method to be overridden, we declare it as final.

```
class Parent {
   // Can't be overridden
   final void show() {}
}

class Child extends Parent {
   // This would produce error
   void show() {}
}
```

**Output**

```
13: error: show() in Child cannot override show() in Parent
   void show() {  }
        ^
  overridden method is final
```

**3. Static methods can not be overridden(Method Overriding vs Method Hiding):**

When you define a static method with the same signature as a static method in the base class, it is known as method hiding. The following table summarizes what happens when you define a method with the same signature as a method in a super-class.

| | **Superclass Instance Method** | **Superclass Static Method** |
|---|---|---|
| **Subclass Instance Method** | Overrides | Generates a compile-time error |
| **Subclass Static Method** | Generates a compile-time error | Hides |

**Super Keyword in Java**

The **super** keyword in Java is a reference variable which is used to refer immediate parent class object. Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

**Usage of Java super Keyword**

super can be used to refer immediate parent class instance variable.
super can be used to invoke immediate parent class method.
super() can be used to invoke immediate parent class constructor.

**1) super is used to refer immediate parent class instance variable.**

We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same fields.

```
class Animal{
String color="white";
}
class Dog extends Animal{
String color="black";
void printColor(){
System.out.println(color);//prints color of Dog class
System.out.println(super.color);//prints color of Animal class
}
```

```
}
class TestSuper1{
public static void main(String args[]){
Dog d=new Dog();
d.printColor();
}}
```

Output:
black
white

In the above example, Animal and Dog both classes have a common property color. If we print color property, it will print the color of current class by default. To access the parent property, we need to use super keyword.

**2) super can be used to invoke parent class method**

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void eat(){System.out.println("eating bread...");}
void bark(){System.out.println("barking...");}
void work(){
super.eat();
bark();
}
}
class TestSuper2{
public static void main(String args[]){
Dog d=new Dog();
d.work();
}}
```

Output:
eating...
barking...

In the above example Animal and Dog both classes have eat() method if we call eat() method from Dog class, it will call the eat() method of Dog class by default because priority is given to local.
To call the parent class method, we need to use super keyword.

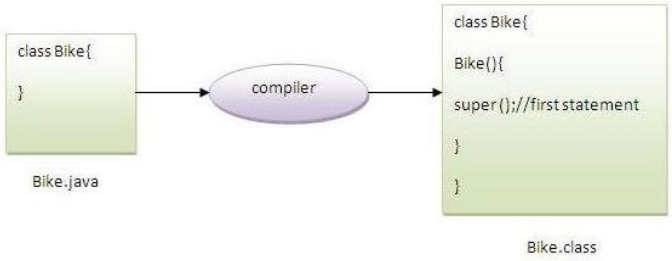
**3) super is used to invoke parent class constructor.**

The super keyword can also be used to invoke the parent class constructor. Let's see a simple example:

```
class Animal{
Animal(){System.out.println("animal is created");}
}
class Dog extends Animal{
Dog(){
super();
System.out.println("dog is created");
}
}
class TestSuper3{
public static void main(String args[]){
Dog d=new Dog();
}}
```

**Output:**
animal is created
dog is created

**Note: super() is added in each class constructor automatically by compiler if there is no super() or this().**




As we know well that default constructor is provided by compiler automatically if there is no constructor. But, it also adds super() as the first statement.

**Another example of super keyword where super() is provided by the compiler implicitly.**

```
class Animal{
Animal(){System.out.println("animal is created");}
}
class Dog extends Animal{
Dog(){
System.out.println("dog is created");
}
}
class TestSuper4{
public static void main(String args[]){
Dog d=new Dog();
}}
```

Output:
animal is created
dog is created

**super example: real use**

Let's see the real use of super keyword. Here, Emp class inherits Person class so all the properties of Person will be inherited to Emp by default. To initialize all the property, we are using parent class constructor from child class. In such way, we are reusing the parent class constructor.

```
class Person{
int id;
String name;
Person(int id,String name){
this.id=id;
this.name=name;
}
}
class Emp extends Person{
float salary;
Emp(int id,String name,float salary){
super(id,name);//reusing parent constructor
this.salary=salary;
}
void display(){System.out.println(id+" "+name+" "+salary);}
}
class TestSuper5{
```

```
public static void main(String[] args){
Emp e1=new Emp(1,"ankit",45000f);
e1.display();
}}
```

Output:
1 ankit 45000

**Final Keyword In Java**

The final keyword in java is used to restrict the user. The java final keyword can be used in many context. Final can be:

1. variable
2. method
3. class

The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in the static block only. We will have detailed learning of these. Let's first learn the basics of final keyword.

**1) Java final variable**

If you make any variable as final, you cannot change the value of final variable(It will be constant).
Example of final variable
There is a final variable speed limit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

```
class Bike9{
 final int speedlimit=90;//final variable
 void run(){
  speedlimit=400;
 }
 public static void main(String args[]){
 Bike9 obj=new  Bike9();
 obj.run();
 }
}//end of class
```

*Output:Compile Time Error*

**2) Java final method**

If you make any method as final, you cannot override it.

```
class Bike{
  final void run(){System.out.println("running");}
}

class Honda extends Bike{
   void run(){System.out.println("running safely with 100kmph");}

   public static void main(String args[]){
   Honda honda= new Honda();
   honda.run();
   }
}
```

*Output:Compile Time Error*

**3) Java final class**

If you make any class as final, you cannot extend it.

```
final class Bike{}

class Honda1 extends Bike{
  void run(){System.out.println("running safely with 100kmph");}

  public static void main(String args[]){
  Honda1 honda= new Honda1();
  honda.run();
  }
}
```

*Output:Compile Time Error*

Q) Is final method inherited?
Ans) Yes, final method is inherited but you cannot override it. For Example:

```
class Bike{
  final void run(){System.out.println("running...");}
}
class Honda2 extends Bike{
   public static void main(String args[]){
    new Honda2().run();
   }
}
```

*Output:running...*

In Java, a **finalizer** is a method that allows an object to perform cleanup operations before the garbage collector destroys it

***Final, finalize, and finalizer are related terms used in programming languages, and they have different purposes:***

**Final**

A keyword used to restrict access to a class, method, or variable. It can be used to create constants, prevent method overriding, or make a class non-inheritable.

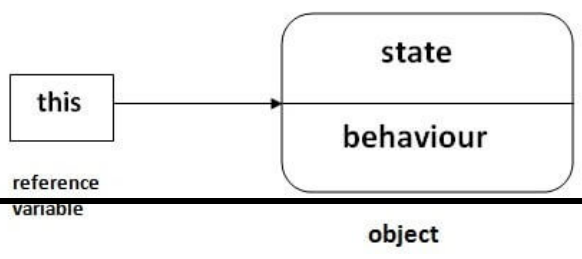
**Finalize**

A method that performs cleanup tasks before an object is garbage collected. In Java, the finalize method is called by the garbage collector.

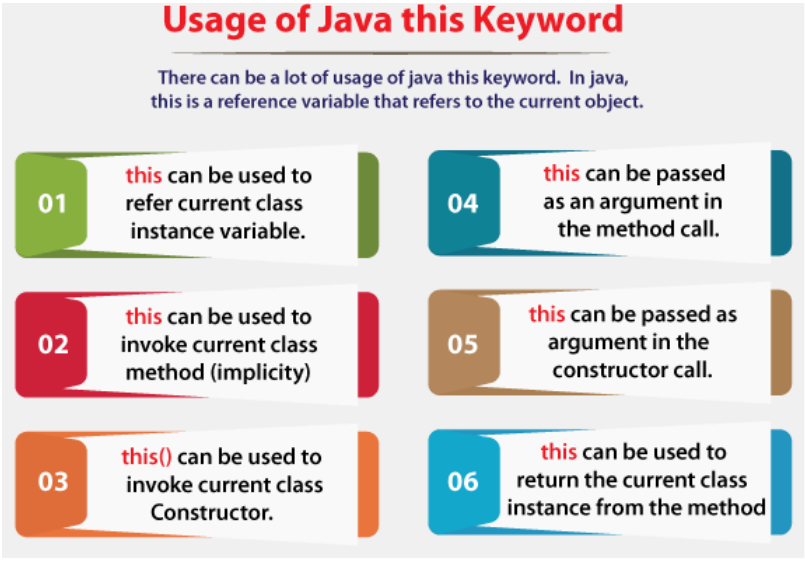
**Finalizer**

A method that performs final clean-up and frees unmanaged resources, such as files, network connections, and windows. Finalizers are also known as destructors. They are invoked automatically and cannot be called.

## this keyword in Java




**There can be a lot of usage of** Java this keyword**. In Java, this is a** reference variable **that refers to the current object.**




## Java Abstraction

Abstract Classes and Methods

Data **abstraction** is the process of hiding certain details and showing only essential information to the user.
Abstraction can be achieved with either **abstract classes** or **interfaces**
The abstract keyword is a non-access modifier, used for classes and methods:
**Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

**Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).
An abstract class can have both abstract and regular methods:

```
abstract class Animal {
  public abstract void animalSound();
  public void sleep() {
    System.out.println("Zzz");
  }
}
```

- From the example above, it is not possible to create an object of the Animal class:
- Animal myObj = new Animal(); // will generate an error
- To access the abstract class, it must be inherited from another class. Let's convert the Animal class we used in the Polymorphism chapter to an abstract class:
- Remember from the Inheritance chapter that we use the extends keyword to inherit from a class.

```
// Abstract class
abstract class Animal {
  // Abstract method (does not have a body)
  public abstract void animalSound();
  // Regular method
  public void sleep() {
    System.out.println("Zzz");
  }
}

// Subclass (inherit from Animal)
class Dog extends Animal {
  public void animalSound() {
    // The body of animalSound() is provided here
```

```
    System.out.println("The dog says: barking");
  }
}
class Main {
  public static void main(String[] args) {
    Dog d= new Dog(); // Create a Dog object
    d.animalSound();
    d.sleep();
  }
}
```

Output:

```
The dog says: barking
Zzz
```

Why And When To Use Abstract Classes and Methods?
To achieve security - hide certain details and only show the important details of an object.
**Note:** Abstraction can also be achieved with Interfaces,

## Java Interface

Interfaces Another way to achieve abstraction in Java, is with interfaces.
An interface is a completely "**abstract class**" that is used to group related methods with empty bodies:

```
// interface
void run(); // interface method (does not have a body)
} interface Animal {
  public void animalSound(); // interface method (does not have a body)
  public void run(); // interface method (does not have a body)
}
```

To access the interface methods, the interface must be "implemented" (kind a like inherited) by another class with the implements keyword (instead of extends). The body of the interface method is provided by the "implement" class:

**Example:**

```
// Interface
interface Animal {
  public void animalSound(); // interface method (does not have a body)
  public void sleep(); // interface method (does not have a body)
}
// Dog "implements" the Animal interface
class Dog implements Animal {
  public void animalSound() {
    // The body of animalSound() is provided here
    System.out.println("The dog says: barking ");
  }
  public void sleep() {
    // The body of sleep() is provided here
    System.out.println("Zzz");
  }
}

class Main {
  public static void main(String[] args) {
    Dog d = new Dog();  // Create a Pig object
    d.animalSound();
    d.sleep();
```

```
  }
}
```
Output:
```
The dog says: barking
Zzz
```

**Notes on Interfaces:**

- Like **abstract classes**, interfaces **cannot** be used to create objects (in the example above, it is not possible to create an "Animal" object in the MyMainClass)
- Interface methods do not have a body - the body is provided by the "implement" class
- On implementation of an interface, you must override all of its methods
- Interface methods are by default abstract and public
- Interface attributes are by default public, static and final
- An interface cannot contain a constructor (as it cannot be used to create objects)

**Why And When To Use Interfaces?**

1) To achieve security - hide certain details and only show the important details of an object (interface).
2) Java does not support "multiple inheritance" (a class can only inherit from one superclass). However, it can be achieved with interfaces, because the class can **implement** multiple interfaces. **Note:** To implement multiple interfaces, separate them with a comma (see example below).

**Multiple Interfaces**

To implement multiple interfaces, separate them with a comma:

Example:

```
interface FirstInterface {
  public void myMethod(); // interface method
}
interface SecondInterface {
  public void myOtherMethod(); // interface method
}
class DemoClass implements FirstInterface, SecondInterface {
  public void myMethod() {
    System.out.println("Some text..");
  }
  public void myOtherMethod() {
    System.out.println("Some other text...");}}

class Main {
  public static void main(String[] args) {
    DemoClass myObj = new DemoClass();
    myObj.myMethod();
    myObj.myOtherMethod();
  }
}
```

Output:

```
Some Text…
Some Other Text…
```